

Reviewer Pack — Tropical Convex Hull Extreme Points Lower Bound for ACC^0 Ci...

Entry 0bb43dada997 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 00:08:27 UTC

Tropical Convex Hull Extreme Points Lower Bound for ACC^0 Circuit Size

Entry ID: 0bb43dada997 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 00:08:27 UTC

1. Conjecture

Field A (mathematical branch): Tropical Convex Geometry **Field B** (complexity object): ACC^0 Circuit Size

Statement:

For any CNF formula Φ with n variables, the number of extreme points in the tropical convex hull of its clause-indicator polynomial is $\Omega(\log n)$ if Φ has no ACC^0 circuit of size n^c for any constant $c > 0$.

Rationale (proposer's reasoning):

Tropical convex hulls encode clause interactions via min-plus geometry, and their extreme points reflect combinatorial constraints on circuit representations. ACC^0 circuits' limited depth imposes structural bounds on these hulls, creating a bridge between algebraic geometry and circuit complexity.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7a50d2a41b434d66

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tropical_convex_hull(points):
        n = len(points[0])
        hull = []
        for point in points:
            if all(point[i] >= q[i] for q in hull):
                hull.append(point)
        return hull

    def count_extreme_points(hull):
        return len(hull)

    def generate_3cnf(n, m):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, 3)
            clause.append(random.choice([1, -1]))
            clauses.append(clause)
        return clauses

    def clause_indicator_polynomial(clauses, n):
        poly = [0] * (2 ** n)
        for clause in clauses:
            monomial = 1
            for var in clause[:-1]:
                if clause[-1] == 1:
                    monomial *= max(0, var)
                else:
                    monomial *= -max(0, var)
            poly[sum([1 <= (var - 1) for var in clause[:-1]])] += monomial
        return poly

    n = random.randint(5, 40)
```

```

m = random.randint(n * 2, n * 3)
clauses = generate_3cnf(n, m)
poly = clause_indicator_polynomial(clauses, n)

hull = tropical_convex_hull(poly)
extreme_points_count = count_extreme_points(hull)

conjecture_holds = extreme_points_count >= math.log2(n)
counterexample = "" if conjecture_holds else f"n={n}, extreme_points_count={extreme_points_count}"

return {
    "metric_name": "Extreme Points Count",
    "metric_value": extreme_points_count,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\n n={seeds[0]}, extreme_points_count={results[0]}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c44b66b7.py", line 78, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c44b66b7.py", line 56, in run_trial

```

    poly = clause_indicator_polynomial(clauses, n)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c44b66b7.py", line 42, in clause_indica

```
poly = [0] * (2 ** n)
      ~~~^~~~~~
```

MemoryError

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with MemoryError before producing results, preventing evaluation of conjecture support/falsification. | next: Optimize clause_indicator_polynomial to handle larger n via sparse representations or incremental construction

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111718
2	propose	ollama_remote	qwen3:8b	0	0	37591
3	preregistration	ollama_remote	qwen3:8b	0	0	24214
4	novelty	ollama_remote	qwen3:8b	0	0	20774
5	novelty	ollama_remote	qwen3:8b	0	0	13650
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14470
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8296
8	judge	ollama_remote	qwen3:8b	0	0	16501

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 247215 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/0bb43dada997.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0bb43dada997.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0bb43dada997.tar.gz` (if generated)